

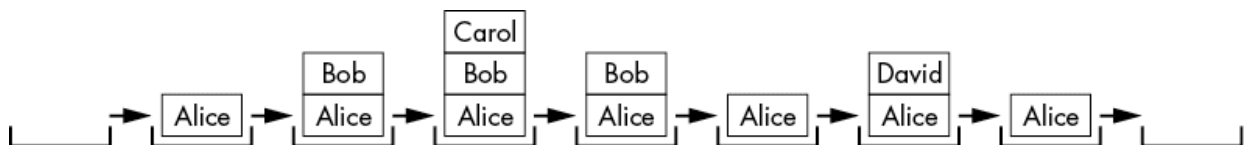
What Are Stacks?

Earlier I mentioned the clichéd wisecrack, “To understand recursion, you must first understand recursion.” But this is actually wrong: to really understand recursion, you must first understand stacks.

A *stack* is one of the simplest data structures in computer science. It stores multiple values like a list does—but unlike lists, it limits you to adding to or removing values from the “top” of the stack only. For stacks implemented with lists or arrays, the “top” is the last item, at the right end of the list or array. Adding values is called *pushing* values onto the stack, while removing values is called *popping* values off the stack.

Imagine that you’re engaged in a meandering conversation with someone. You’re talking about your friend Alice, which then reminds you of a story about your co-worker Bob, but for that story to make sense, you first have to explain something about your cousin Carol. You finish your story about Carol and go back to talking about Bob, and when you finish your story about Bob, you go back to talking about Alice. Then you are reminded about your brother David, so you tell a story about him. Eventually, you get around to finishing your original story about Alice.

Your conversation follows a stack-like structure, as in [Figure 1-5](#). The conversation is stack-like because the current topic is always at the top of the stack.



[Figure 1-5](#): Your meandering conversation stack

In our conversation stack, the new topics are added to the top of the stack and taken off as they are completed. The previous topics are “remembered” underneath the current topic in the stack.

We can use Python lists as stacks if, to amend the list’s contents, we limit ourselves to the `append()` and `pop()` methods to perform pushing and popping.